# ADR 0002
# Controller - First

| Data | Status | Integrantes |
|---|---|---|
| 05/02/2026 | Aceito | Alexander Nogueira<br>Caio Nunes de Abreu<br>Cassiano Nunes de Abreu<br>Gabriel Naoki Uto Turigoe<br>Wyllian Mariano Nogueira |

## Sumário


Contexto 2
Decisão 2
Consequências 2
- Positivas 2
- Negativas / Riscos 3

Mitigações 3
Alternativas consideradas 3
Follow Up / Próximos passos 3
Referencias 3

# Contexto

A aplicação **Agenda de Contatos** (Java 21 / Swing) apresentava lógica de validação, regras de negócio e persistência misturadas diretamente nas classes de interface gráfica (Views), como `Form_Cadastro` e `Form_Listagem`. Essa estrutura causava:

- Dificuldade na criação de testes automatizados.
- Alto acoplamento entre a UI e a forma de salvar os dados.
- Impedimento para uma evolução organizada da arquitetura do software.

O objetivo é promover a separação de responsabilidades de forma gradual, ensinando princípios como alta coesão e baixo acoplamento sem complexidade excessiva inicial.

# Decisão

Adotar a estratégia **"Controller-first"** como um refactoring incremental:

- **Interfaces:** Criar o pacote `controllers` com as interfaces `FormController` (para cadastros) e `ListController` (para listagens).
- **Implementação Concreta:** Implementar a classe `ContatoController`, que inicialmente acumulará a lógica de negócio e a persistência em arquivo texto (`agenda.txt`) via `java.nio.file`.
- **Injeção de Dependência:** O `Path` do arquivo deve ser injetável para permitir testes com arquivos temporários.
- **Desvinculação da UI:** As Views devem depender apenas das interfaces, e todas as operações de I/O devem rodar em background via `SwingWorker` para não travar a interface (EDT).

# Consequências

## Positivas

- **Testabilidade:** Possibilidade de testar a lógica de negócio independentemente da interface gráfica.
- **Organização:** Redução do acoplamento e melhor definição das responsabilidades de cada componente.
- **Evolução:** Facilita a futura migração para camadas de `Repository` ou `Service`.

## Negativas / Riscos

- **Antipattern:** Risco de manter a lógica de persistência "pendurada" no Controller por tempo demais.
- **Violação de Camadas:** O Controller assumirá temporariamente o papel que seria de um repositório.
- **Limitação Técnica:** O uso de arquivos `.txt` possui riscos de concorrência e segurança.

# Mitigações

- Criação de uma issue técnica para extrair a camada de **Repository** no futuro (até a entrega N2).
- Garantir que os métodos do Controller sejam pequenos e bem nomeados para facilitar a substituição posterior.
- Uso obrigatório de `SwingWorker` para proteger a experiência do usuário na UI.
-

# Alternativas consideradas

- Atualizar o Documento de Arquitetura de Software (DAS) e os diagramas C2/C3.
- Implementar no mínimo 3 testes unitários para o Controller.
- Garantir que as Views deleguem as ações exclusivamente para as interfaces dos controllers.
-

# Follow Up / Próximos passos

- Criar issue tracking (Se estivéssemos utilizando uma ferramenta para a gestão do projeto):
  - #Extrair-Repository-Post-N1 com checklist de tarefas (definir interface Repository, mover I/O do Controller, atualizar testes e ADR).
- Atualizar DAS e diagramas (C3/C2) para refletir Controller-first e, quando a extração ocorrer, atualizar ADR para registrar migração.
- Exigir entregas: controllers com Path injetável, testes unitários do Controller (mín. 3), Views delegando à interface e uso de SwingWorker para I/O.

# Referências

- Repositório do Projeto: https://github.com/marquesclayton/ads3.contatos
- Conceito: Strangler Fig Pattern (Martin Fowler)